

Relatório Final do Trabalho Prático

Desenvolvimento de Aplicação Android: MealsApp

UC Sistemas Móveis e Aplicações

Miguel Grilo^{1*} and Tiago Ramalho^{2*}

^{1*}Inteligência Artificial e Ciências de Dados, Universidade de Évora,
Évora, Portugal.

^{2*}Engenharia Informática, Universidade de Évora, Évora, Portugal.

*Corresponding author(s). E-mail(s): {158387,
158514}@alunos.uevora.pt;

1 Breve Descrição da Aplicação

A **MealsApp** é a app ideal para descobrir, pesquisar e explorar receitas de todo o mundo de forma rápida e intuitiva. A aplicação foi desenhada com foco na usabilidade e na performance, oferecendo as seguintes funcionalidades principais:

- Descoberta aleatória de pratos populares exibidos num ecrã principal dinâmico.
- Pesquisa avançada por nome e filtros intuitivos por categorias.
- Detalhes da receita: imagens de alta qualidade, lista de ingredientes com medidas exatas, instruções passo-a-passo e um atalho direto para o tutorial em vídeo no YouTube.
- Suporte total a *Dark Mode* e uma adaptação fluida a qualquer formato de ecrã (telemóvel ou tablet, orientação vertical ou horizontal).
- Experiência resiliente, garantindo um tratamento elegante de estados de carregamento (*Loading*) e de erros de conectividade.

2 Mapeamento: Conceitos Lecionados e Desenvolvimento

O desenvolvimento da *MealsApp* aplicou rigorosamente os conceitos fundamentais do programa *Android Basics with Compose*.

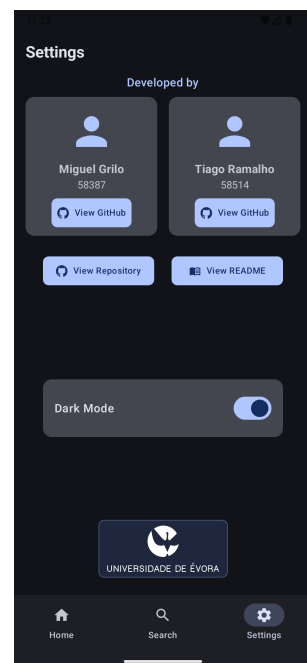
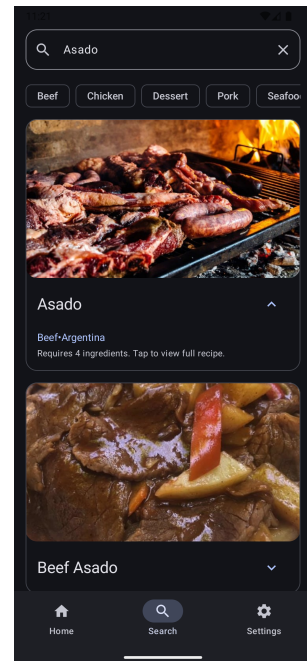
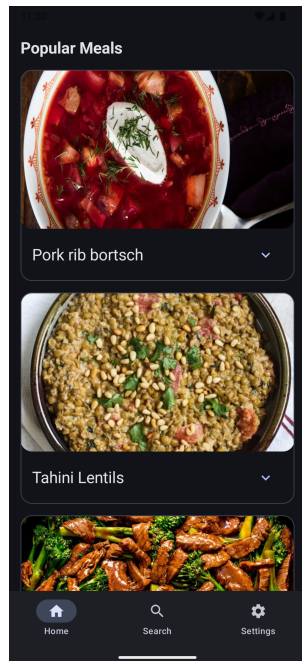


Fig. 1 Demonstração da interface: Ecrã Home, Search, Details e Settings

2.1 Unidade 1 e 2: Layouts Básicos, Estado e Interatividade

A interface de utilizador foi desenhada de forma declarativa, recorrendo a *composables* como `Text` e `Image`, altamente personalizados com `Modifiers`. Implementou-se o **estado local** (`mutableStateOf`) em componentes como o `ExpandableMealCard` para gerir a revelação de detalhes. Aplicou-se o princípio de *State Hoisting* de forma global (elevando o evento `onQueryChange` da `SearchScreen` para o respetivo `ViewModel`), garantindo que os componentes visuais permanecessem independentes de lógica (*stateless*).

2.2 Unidade 3: Listas e Material Design

A apresentação eficiente de conjuntos de dados foi alcançada através do `LazyVerticalGrid`, criando uma grelha de receitas perfeitamente responsiva. A aplicação adota integralmente as normas do **Material Design 3**, com a centralização de tipografia, formas e paletas de cores dinâmicas no `Theme.kt`. Garantimos a **Acessibilidade** através da atribuição sistemática de `contentDescription` aos ícones e validação semântica das ações (`assertHasClickAction`) nos nossos testes, otimizando o suporte para leitores de ecrã (*TalkBack*).

2.3 Unidade 4: Navegação e Arquitetura

O projeto segue o padrão **MVVM** (Model-View-ViewModel) alinhado com os princípios de *Clean Architecture*, segregando claramente as responsabilidades entre as camadas `data`, `model` e `ui`.

A gestão de dependências foi implementada através de um `AppContainer` manual. Esta escolha estratégica, em detrimento de *frameworks* de injeção de dependências, foi motivada pela necessidade de garantir uma testabilidade superior; ao injetar as instâncias manualmente, conseguimos substituir facilmente os repositórios reais por implementações de teste (como o `FakeMealsRepository`), permitindo a criação de testes unitários puros, isolados de componentes externos e com execução rápida.

A navegação entre os ecrãs principais é orquestrada pelo componente `NavHost`, com especial atenção à criação de **layouts adaptativos** suportados por *WindowSizeClasses*. Estes permitem uma transição inteligente entre uma `NavigationBar` em dispositivos móveis e uma `NavigationRail` em orientações panorâmicas ou tablets. Adicionalmente, a aplicação assegura a persistência de estado para as preferências do utilizador, como o *Dark Mode*, através de uma implementação `InMemoryPreferencesRepository` que mantém a consistência da experiência durante todo o ciclo de vida da aplicação.

2.4 Unidade 5: Ligações à Internet

A obtenção de dados em tempo real da *TheMealDB API* foi implementada com a biblioteca **Retrofit** e `kotlinx.serialization`. Para assegurar o desempenho, o processamento de rede e a transformação de dados decorrem de forma assíncrona recorrendo a **Corrotinas** (`viewModelScope`). O carregamento e armazenamento em *cache* de imagens é efetuado de forma nativa com a biblioteca **Coil** (`AsyncImage`). Todos os

fluxos estão protegidos por uma rigorosa gestão de **Estados da UI** (*Loading, Success, Error*).

3 Obstáculos Encontrados

Durante a execução do projeto, superámos diversos desafios técnicos relevantes:

- **Estrutura e Limitações da API:** A base de dados externa fornece os ingredientes e respetivas medidas distribuídos por 40 campos estáticos e distintos (ex: `strIngredient1`).

Resolução: Desenvolvemos uma lógica de mapeamento dinâmica no `MealMapper` para iterar sobre a resposta JSON, descartando elementos nulos e strings vazias, consolidando a informação em listas limpas prontas a serem consumidas pela UI.

- **Performance e Obtenção Múltipla de Dados:** O *endpoint* para receitas aleatórias da API suporta a devolução de apenas uma receita por pedido, causando potenciais *bottlenecks* de velocidade ao carregar a *HomeScreen*.

Resolução: Utilizámos paralelismo avançado com Corrotinas. Instanciando processos concorrentes com `async` e centralizando a sua finalização com `awaitAll`, disparámos 8 pedidos HTTP simultaneamente, reduzindo significativamente os tempos de carregamento para o utilizador final.

4 Objetivos Atingidos

- **Maturidade do Produto:** Construção de uma aplicação Android robusta, de arquitetura moderna e plenamente funcional, pronta para cenários de produção.
- **Cumprimento Curricular:** Adoção prática e consolidação de todas as diretrizes requeridas pelo curso *Android Basics with Compose*.
- **Fiabilidade e Qualidade:** Implementação de uma sólida grelha de testes (100% dos *ViewModels* e Mapeadores testados) alavancada por Testes Instrumentados focados no comportamento e acessibilidade da UI.
- **Boas Práticas de Software:** Concretização de código escalável, limpo e modularizado, valorizando as fundações exigidas a nível empresarial na engenharia de *software mobile*.

5 Conclusão

O desenvolvimento da MealsApp permitiu consolidar a aplicação de padrões de arquitetura modernos com as bibliotecas mais recentes do ecossistema Android. A modularização do código facilitou a implementação de uma estratégia de testes robusta, assegurando que tanto a lógica de negócio como a interface de utilizador apresentam um comportamento previsível e acessível. Este projeto demonstra a viabilidade de construir aplicações Android complexas, escaláveis e com foco na qualidade de experiência do utilizador final.